SECUREX LABS
OFFENSIVE SECURITY DIVISION

RED TEAM ENGAGEMENT

TRY HACK ME

Try Hack Me

LEARN. PRACTICE. HACK. REPEAT.

FULL PENETRATION TEST REPORT

- **INFORMATION GATHERING** — Open Source Intelligence
- **VULNERABILITY ANALYSIS** — Deep Discovery & Enumeration
- **EXPLOITATION** — Attack Vector Validation
- **PRIVILEGE ESCALATION** — Access & Control
- **IMPACT & REPORTING** — Business Risk & Remediation



PROFESSIONAL SECURITY TESTING | REAL WORLD SCENARIOS | ATTACKER MINDSET DEFENDER VIEW | ACTIONABLE INSIGHTS MAXIMUM IMPACT

CONFIDENTIAL
FOR AUTHORIZED USE ONLY

CONFIDENTIAL • SENSITIVE INFORMATION • DO NOT DISTRIBUTE

Assessment scope 2
Summary 3
Overview 3
Results 3
Impact 3
Remediation 3
Vulnerabilities Writeups 3
Title: 4
Risk Rating: 4
Summary 4
Background 4
Technical Details and Evidence 4
Impact: 6
Remediation: 6
Title: 7
Risk rating: 7
Summary: 7
Background: 7
Technical Details and Evidence 7
Impact: 8
Remediation: 8
Title: 8
Risk rating: 8
Summary: 9
Background: 9
Technical Evidence and Details 9
Impact: 10
Remediation: 10
Title: 10
Risk rating: 10
Summary: 10

Background: 10
Technical Evidence and Details 10
Impact: 13
Remediation: 13
Title: 13
Risk rating 13
Technical Evidence and Details 14
Impact: 14
Remediation: 14

Assessment scope

A black-box penetration test was conducted against the provided virtual environment, covering external networks, web applications, and internal assessments.

The assessment was limited to the assigned IP address. All tools and techniques were permitted. The objective was to identify and document vulnerabilities, demonstrate exploitation, and obtain the User.txt and Root.txt flags as evidence of compromise. The host file was also modified as required to reflect the internal environment.

Summary

Overview

An authorized black-box penetration test was conducted against the Internal environment to identify vulnerabilities across its web application, internal services, and Linux infrastructure.

Results

The assessment successfully demonstrated a complete attack chain from external web access to root-level compromise. Key findings included weak WordPress authentication, PHP code execution through the WordPress theme editor, plaintext credential exposure, insecure Jenkins access, and excessive administrative privileges.

Impact

An attacker could leverage these weaknesses to obtain unauthorized access, execute commands remotely, compromise user accounts, access internal services, and ultimately obtain full control of the underlying host.

Remediation

Remediation should focus on enforcing strong passwords and multi-factor authentication, disabling WordPress theme and plugin code editing, removing plaintext credentials, and rotating all compromised passwords. Jenkins' access should be restricted, and the Script Console should be secured. Network segmentation and firewall controls should also be implemented to limit unauthorized access between systems. Finally, direct root SSH access should be disabled, and least-privilege principles should be enforced across the environment.

Vulnerabilities Writeups

Title: Weak WordPress Credentials

Risk Rating: Critical

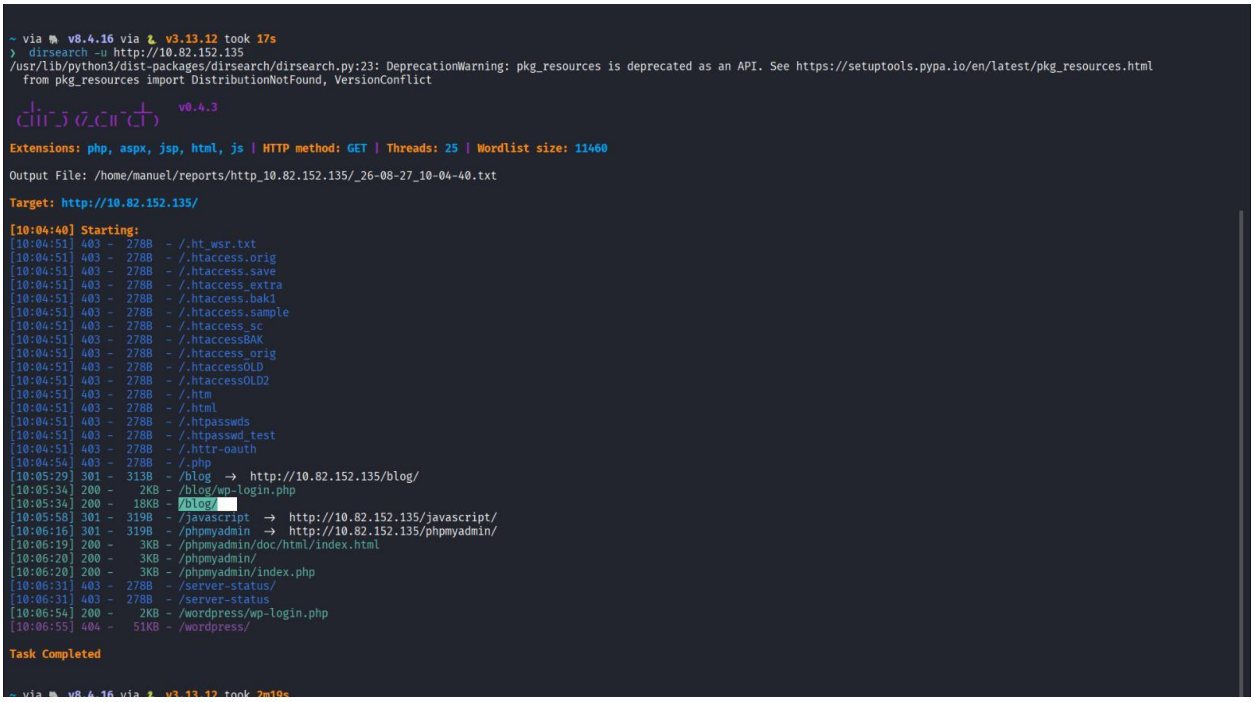
Summary

Weak WordPress credentials allowed password guessing to compromise a valid account, resulting in unauthorized access to the WordPress administrative interface.

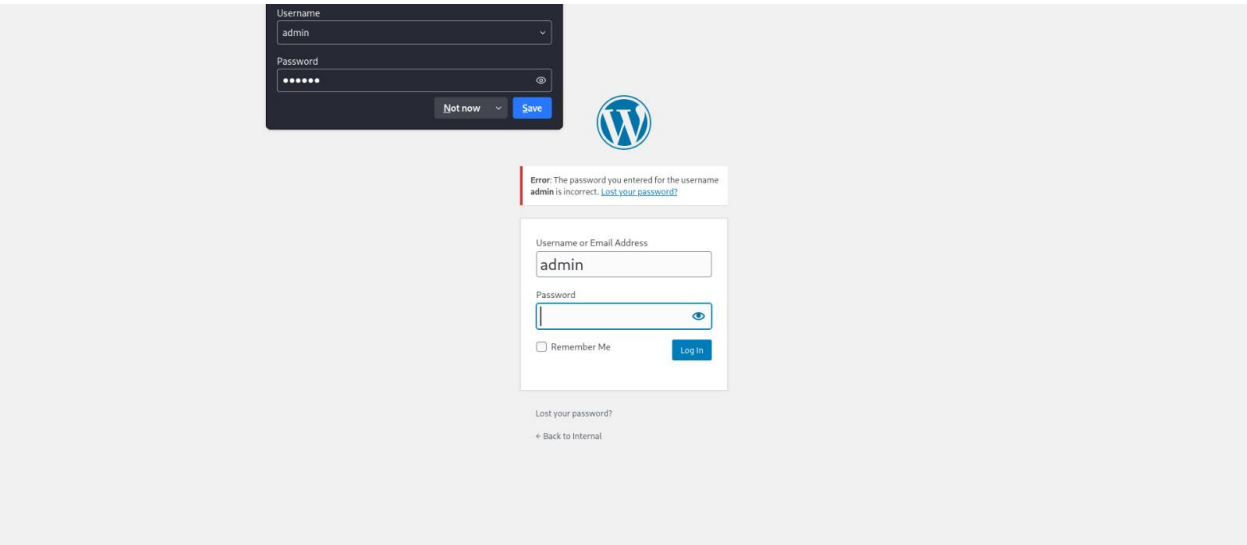
Background

Weak WordPress credentials could allow an attacker to successfully guess valid user credentials through password attacks.

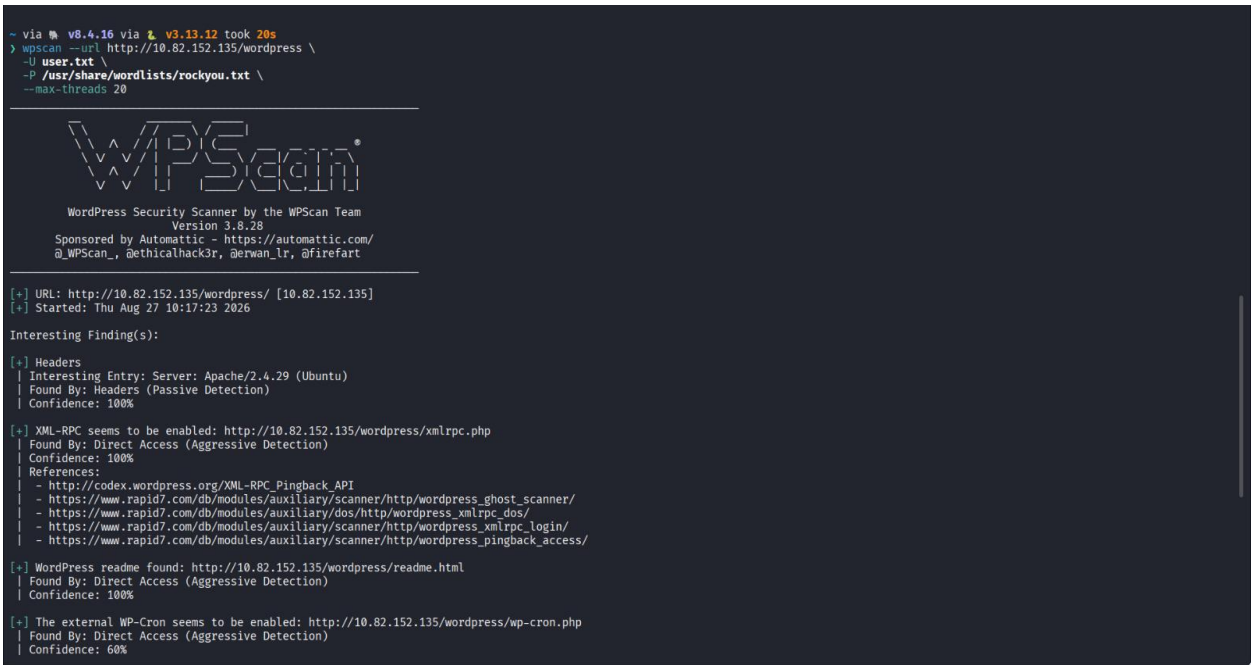
Technical Details and Evidence

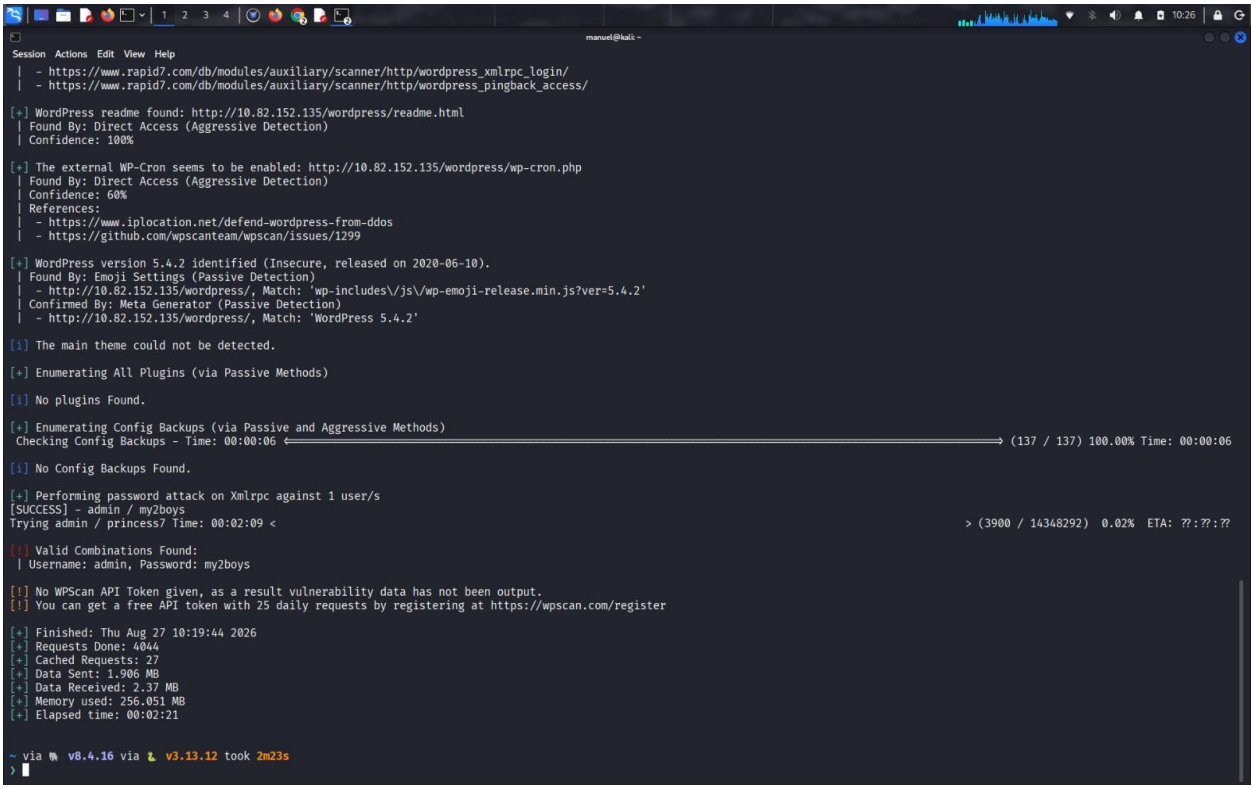


Multiple of the directories were found on the application. This led to a login page of WordPress.



Given the response of the password for the admin is wrong, this shows that the admin user existed. This allows a brute force attack on the application using the list of called passwords called rockyou.txt.





The above shows that the password was found which is my2boys allowing authentication.

Impact:

Leads to unauthorized access to the WordPress administration panel and potentially access sensitive content or privileged functionality.

Remediation:

Enforce strong, unique passwords, enable MFA, implement login rate limiting/lockout controls, and monitor failed authentication attempts.

Title: WordPress Theme Editor Enabled Administrative access allowed modification of PHP code, leading to Remote Code Execution (RCE).

Risk rating: Critical

Summary:

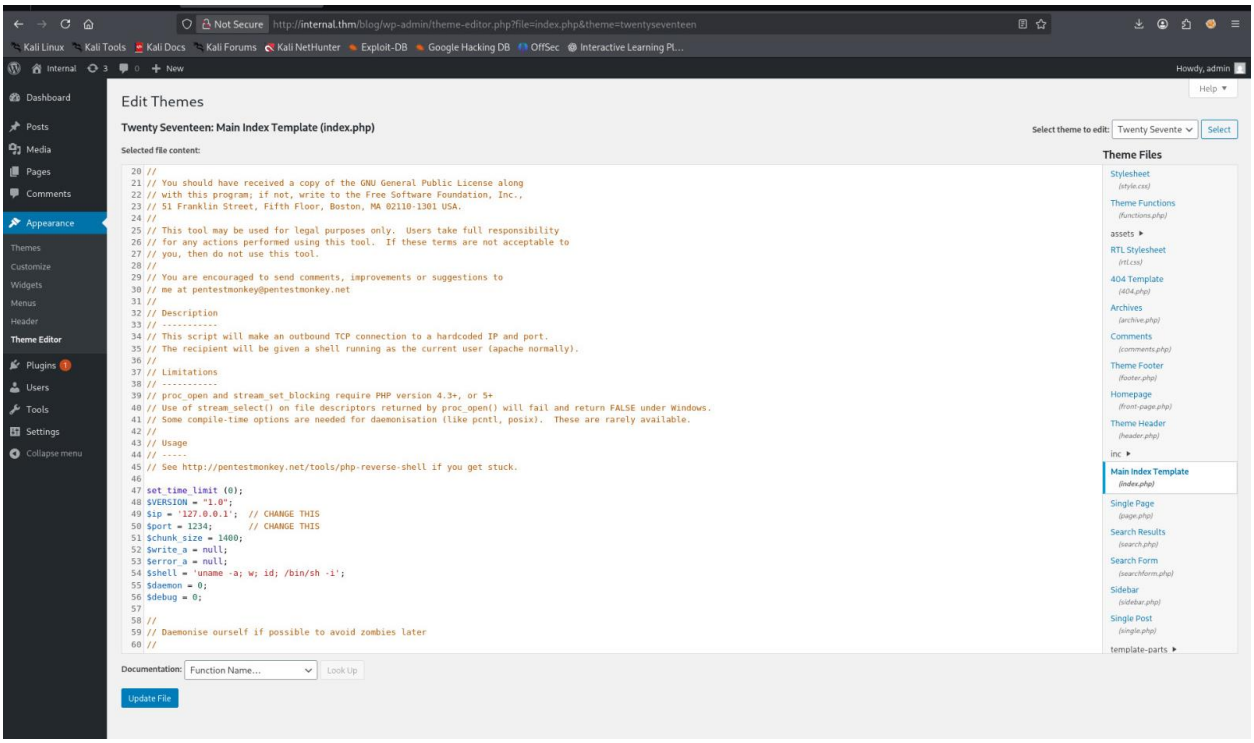
The WordPress Theme Editor was enabled, allowing administrators to directly modify executable PHP files within the active theme.

Background:

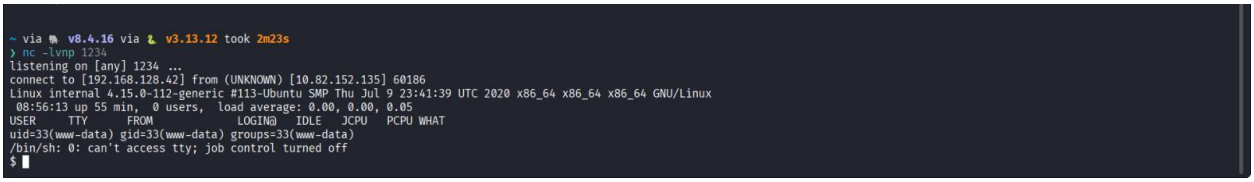
An attacker who gains WordPress administrative access could modify a PHP theme file and insert malicious code. When the affected page is requested, the code could be executed on the server under the web server account.

Technical Details and Evidence

The discovered credentials were successfully used to authenticate to the WordPress administration panel. The panel provided access to multiple administrative editors, including the Theme Editor, which allowed modification of executable PHP files. This functionality was to insert a reverse-shell payload, resulting in remote command execution and a shell on the underlying server.



The above image shows the script.



The above image shows netcat listening on port 1234 just as the configuration of the payload. This shows that the script was executed successfully and the shell was gained.

Impact:

This results in remote code execution, allowing an an access sensitive data, modifying website content, and potentially using the compromised server as a steppingstone for further attacks.

Remediation:

Disable the WordPress Theme/Plugin Editor using DISALLOW_FILE_EDIT and restrict administrative access to trusted users.

Title: Plaintext Credential Exposure

Sensitive usernames/passwords were stored in accessible files such as /opt.

Risk rating: High

Summary:

Sensitive credentials were stored in plaintext within files on the server, making them accessible to users who obtained sufficient filesystem access.

Background:

An attacker who compromises on the server could search accessible directories for plaintext credentials and use them to authenticate to other accounts or services.

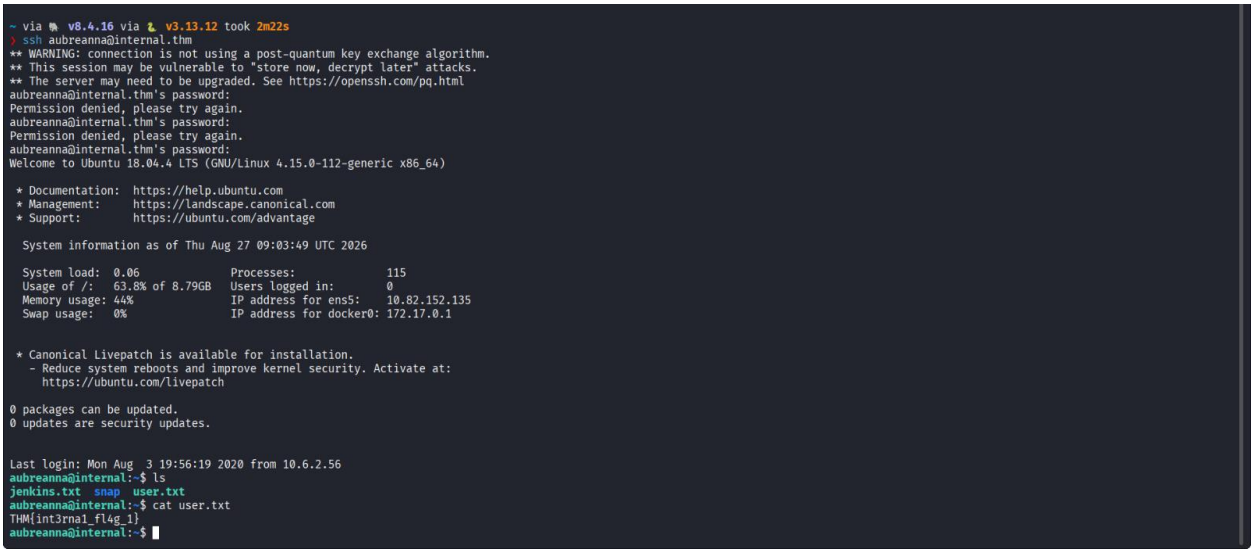
Technical Evidence and Details

After gaining a shell, accessible directories were enumerated for sensitive information. The /opt directory contained a text file exposing usernames and passwords in plaintext, which could be used to authenticate other services or accounts.



The obtained credentials were successfully used to authenticate to the SSH service, providing access to the user account. Successful authentication allowed access to the

user.txt flag, confirming unauthorized access to the account.



Impact:

Exposed credentials could lead to unauthorized access, privilege escalation, lateral movement, and potentially full system compromise.

Remediation:

Remove plaintext credentials from the filesystem, rotate all exposed passwords, use secure secrets management, and enforce strict file permissions.

Title: Jenkins script console access allows access to the Script Console and arbitrary command execution.

Risk rating: Critical

Summary:

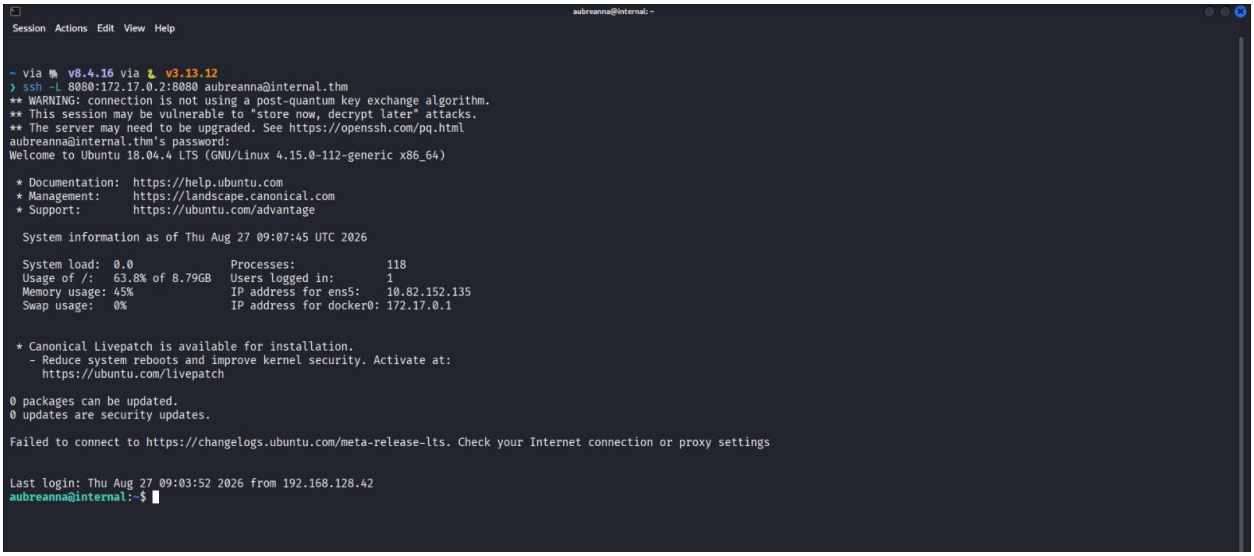
The Jenkins Script Console is accessible to an authenticated user, allowing arbitrary Groovy code to be executed on the Jenkins server.

Background:

An attacker who obtains Jenkins administrative credentials could use the Script Console to execute commands on the underlying system, potentially establishing a shell and accessing sensitive resources.

Technical Evidence and Details

The credentials obtained for the aubreanna account were used to establish an SSH tunnel to the internal Jenkins service. The tunnel exposed the internal Jenkins login page locally, allowing further assessment.

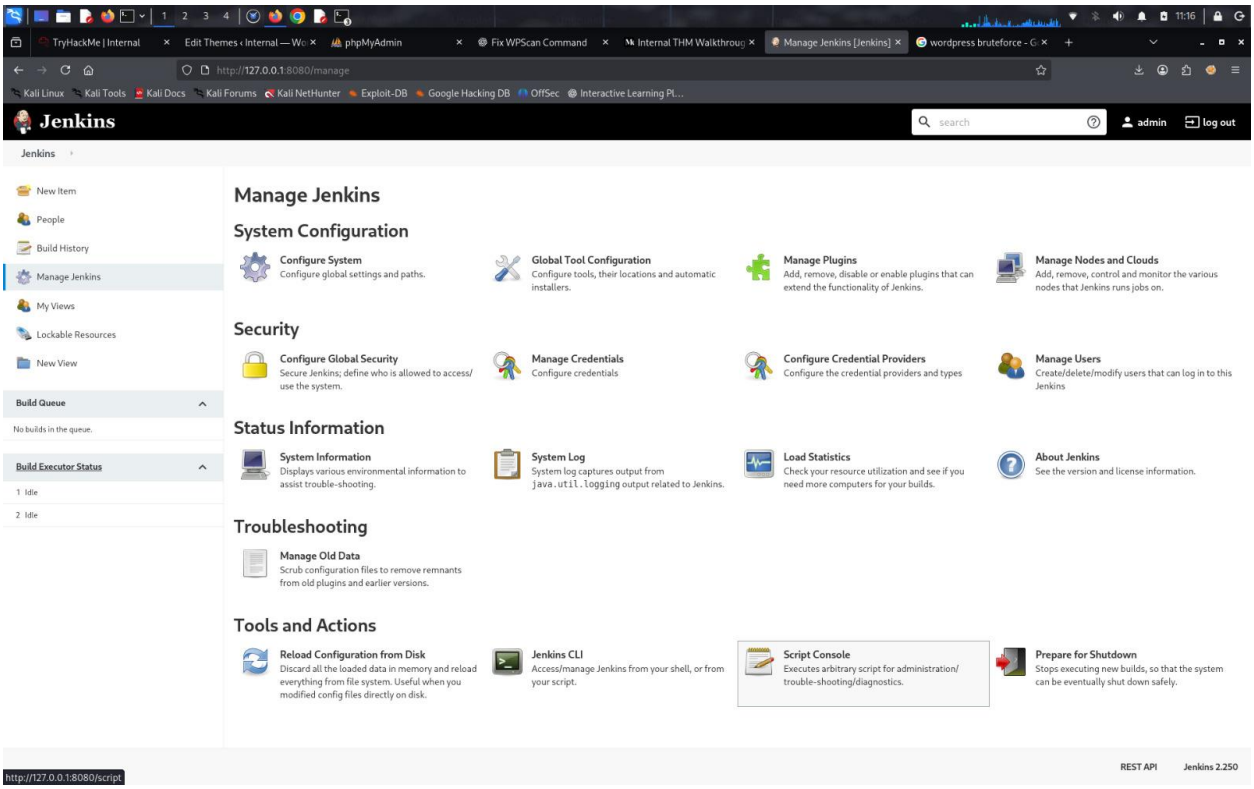


The Jenkins login page was tested using Hydra with the identified jenkins username and successfully obtained the password spongebob

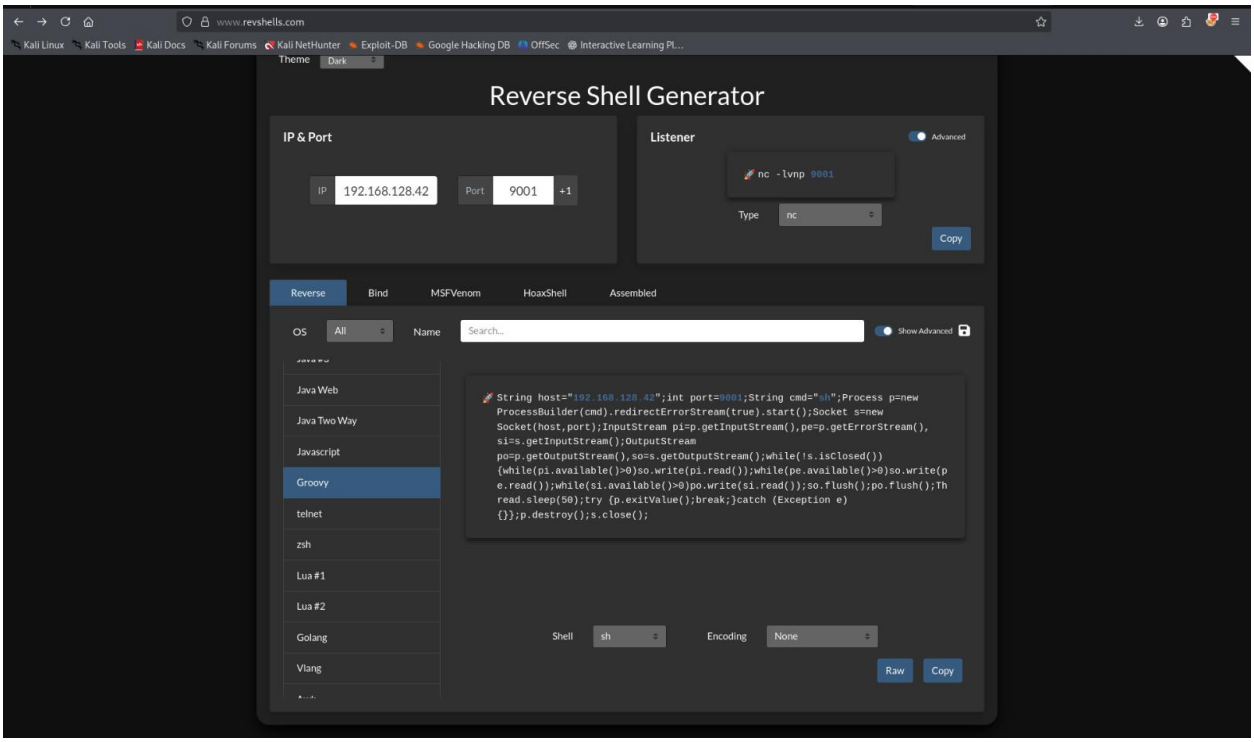




Authenticated access to the Jenkins administration interface provided access to the Script Console.



A Groovy script was executed through the console,



resulting in a successful connection back to the assessment machine and establishing a shell on the Jenkins host.

The obtained shell was verified with:



whoami
jenkins

This confirmed that arbitrary code execution was possible through the Jenkins Script Console under the jenkins service account and by navigating to /opt text was found showing plain text Credentials that were used for further compromising.

Impact:

This results in remote code execution, data exposure, credential theft, privilege escalation, and compromise of the underlying host.

Remediation:

Restrict Script Console access to trusted administrators, enforce strong authentication and MFA, apply least-privilege permissions, and monitor Jenkins for administrative activity.

Title: Excessive Privileges causes compromised accounts could ultimately be leveraged to achieve root-level access.

Risk rating: Critical

Summary: Insufficient privilege of separation allowed a compromised account to be leveraged to obtain root-level access to the system.

Background: An attacker who compromises a lower-privileged account could abuse exposed credentials or excessive permissions to access privileged accounts and execute actions as root.

Technical Evidence and Details

The root credentials discovered on the compromised host were used to authenticate the SSH service as the root user. Successful authentication provided full administrative access to the system. The root.txt flag was subsequently accessed, confirming successful root-level compromise.



Impact:

Successful exploitation results complete system compromise, allowing an attacker to access sensitive data, modify system files, and control the host.

Remediation:

Apply least-privilege principles, restrict privileged account access, remove unnecessary permissions, secure privileged credentials, and disable direct root authentication where possible.